PortfolioOptimisers.jl

Daniel Celis Garza[1]

[1]Independent researcher, Oxford, UK

August 20, 2026

Abstract

Portfolio optimisation is the science of either: *1)* Minimising risk whilst keeping returns to acceptable levels. *2)* Maximising returns whilst keeping risk to acceptable levels. To some definition of acceptable, and with any number of additional constraints available to the optimisation type. There exist myriad statistical, pre- and post-processing, optimisations, and constraints that allow one to explore an extensive landscape of "optimal" portfolios. PortfolioOptimisers.jl is an attempt at providing as many of these as possible under a single banner and making it accessible to all: 57 risk measures, 18 covariance estimators, 12 prior estimators and 16 optimisers, which compose with each other rather than sit in separate silos, and which a single cross-validation framework can tune as one object. We make extensive use of Julia's type system, module extensions, and multiple dispatch to simplify development and maintenance, while keeping robustness, testability, and usability high.

Keywords. Portfolio Optimisation · Quantitative Investment · Conic Optimisation · Parameter Estimation

1 Introduction

The field of portfolio optimisation was introduced by Harry Markowitz's seminal 1952 paper, *Portfolio Selection* [1]. The field has grown enormously since then, and so has the catalogue of that model's failure modes. It attempts to summarise the entire distribution of returns in highly compressed summary statistics, both of which can be sensitive to outliers and atypical conditions for the lookback period. The optimiser then behaves as an error maximiser rather than a diversifier [2], is acutely sensitive to the expected returns it is given [3], and suffers roughly an order of magnitude more from errors in the means than from errors in the covariance [4]. The equal weighted portfolio remains a stubborn out of sample benchmark [5]. Sixty years of responses, from constraints [6] and shrinkage [7] to norm regularisation [8], robust formulations and hierarchical ones, are by now a literature of their own [9], [10], [11], [12].

Unfortunately, most implementations live in disparate, unconnected, often proprietary, and/or bespoke codebases, which limit their applicability. It is only recently that various

advanced yet very usable libraries have been published [13], [14], [15]. However, each has its own strengths, weaknesses, scope, and idiosyncrasies. That is no different from PortfolioOptimisers.jl, but it is our hope that this library serves as a unifying framework for the various techniques and methods available in the field, while also eventually providing a simple and intuitive interface for users, with advanced features for experts.

There also exist myriad optimisation methods, pre-filtering, distribution and moment estimators, machine learning techniques, validation, and parameter tuning methods that can all be used together to improve the out-of sample performance of a portfolio. To this day, only skfolio [15] has succeeded in providing a unified framework for this. PortfolioOptimisers.jl provides an alternative that is not tied to the scikit-learn [16] or cvxpy [17], [18] APIs and ecosystems, as well as providing different functionality and a different architectural philosophy.

Table 1 gives a sense of the scale. What follows is a tour rather than a manual: the documentation carries the detail, and the point of this paper is to show that the detail is there.

2 Design and implementation

2.1 Basic example

PortfolioOptimisers.jl is built with modularity and extensibility in mind. We can demonstrate a simple improvement over the Markowitz model by adding an L2 regularisation term to the optimisation problem, which shrinks the weights towards the equal weighted portfolio and is known to improve out of sample performance [8].

$$\begin{aligned} \min_{\boldsymbol{w}} \quad & \boldsymbol{w}^\top \boldsymbol{\Sigma} \boldsymbol{w} + \lambda \left\| \boldsymbol{w} \right\|_2^2 \\ \text{s.t.} \quad & \boldsymbol{w}^\top \boldsymbol{\mu} \geq \mu_i \\ & \boldsymbol{w}^\top \mathbf{1} = 1 \\ & 0 \leq w_j \leq 1 \quad \forall j \neq \text{AAPL} \\ & 0 \leq w_{\text{AAPL}} \leq 0.2 \end{aligned} \tag{2.2}$$

The problem is solved once per return level μ_i, for $N = 100$ levels evenly spaced between the minimum and maximum attainable returns, which traces the efficient frontier of Figure 2.

```
# Import packages.
using PortfolioOptimisers, CSV, TimeSeries, Clarabel, StatsPlots, GraphRecipes
# Load prices and turn them into returns.
X = TimeArray(CSV.File(joinpath(@__DIR__, "../../examples/SP500.csv.gz"));
```

Table 1: Concrete estimator and algorithm types per family, counted from the library's own type tree; the Result types they return are not counted. The 16 optimisers are 3 naïve, 5 JuMP.jl based, 4 clustering, 2 ensemble, and 2 finite allocators. Composition multiplies them: any covariance estimator feeds any distance, any distance feeds any clustering or network algorithm, and any of those feeds any optimiser that asks for one.

Family	**Types**	**A few of them**
Risk measures	57	Variance, CVaR, EVaR, RLVaR, their range and drawdown twins, ordered weights, high order moments
Covariance and correlation	18	Gerber, Smyth-Broby, Kendall, Spearman, mutual information, distance covariance, lower tail dependence
Expected returns	9	Equilibrium, excess, median, windowed, and shrunk towards three targets
Prior statistics	12	Empirical, factor, high order, four Black-Litterman variants, entropy pooling, opinion pooling
Phylogeny algorithms	14	Three minimum spanning trees, DBHT, hierarchical clustering, k-means, eight centrality measures
Uncertainty sets	4	Normal, ARCH bootstrapped, delta and characteristic, in box or ellipsoidal form
Constraints, costs, and penalties	16	Weight bounds, three budget forms, linear equations, thresholds, risk budgets, factor exposure, two phylogeny forms, centrality, fees, turnover, and L2 or Lp weight penalties. Custom constraints and objectives are an abstract type to subtype, so they are not counted
Optimisers	16	Equal weighted, inverse volatility, mean risk, near optimal centering, risk budgeting, HRP, HERC, Schur, nested clustering, stacking, discrete allocation
Cross-validation and tuning	7	K-fold, combinatorial, walk-forward by index or by date, multiple randomised, grid and randomised search
Preprocessing and pipeline	8	Prices to returns, missing data filter, imputer, three asset selectors, train and test split, pipelines that bundle end-to-end workflows and can be cross-validated and tuned as a single unit

```
                timestamp = :Date)
rd = prices_to_returns(X)
rd_train, rd_test = train_test_split(rd; test_size = 0.2)
# Mean risk optimisation, which minimises risk by default.
mr = MR(;
  # Variance, written directly as a quadratic expression.
  r = Variance(; alg = QuadRiskExpr()),
  opt = JuMPOpt(;
    # Solvers are tried in order until one of them succeeds.
    slv = Solver(; name = :clarabel, solver = Clarabel.Optimizer,
                 settings = Dict("verbose" => false)),
    # An estimator, so the bounds are built from the data: every asset
    # is capped at 1 and floored at 0, except AAPL, capped at 0.2.
    wb = WBE(; ub = "AAPL" => 0.2),
```

```
    # Maps asset names to their columns, and names to sets of assets.
    # Constraint estimators and some priors are built against this.
    sets = UniverseSets(; dict = Dict("nx" => rd.nx)),
    # Squared L2 penalty on the weights, lambda = 1e-4.
    l2 = L2Reg(; val = 0.0001, alg = QuadRiskExpr()),
    # Sweep 100 return levels: one solve each, one efficient frontier.
    ret = ArithmeticReturn(;
            settings = JuMPReturnsSettings(; lb = Frontier(; N = 100)))
  ) # opt
) # mr
# Fit on the training set, then score both sets.
res = optimise(mr, rd_train)
pred_train = predict(res, rd_train)
pred_test = predict(res, rd_test)
# Scenario based standard deviation, as a second order cone expression.
r = SCM(; alg = SOCRiskExpr())
plt = plot_measures(pred_train; x = r, label = "Training", zcolor = nothing)
plt = plot_measures(pred_test; x = r, plt = plt, label = "Test", zcolor = nothing,
                    markercolor = :red, ylabel = "Mean Return",
                    xlabel = "Standard Deviation")
savefig(plt, joinpath(@__DIR__, "fig1.svg"))
nothing
```

Figure 1: End-to-end example of a regularised Markowitz model. The listing is not a transcript: it is executed by the build, so it cannot drift away from the library.

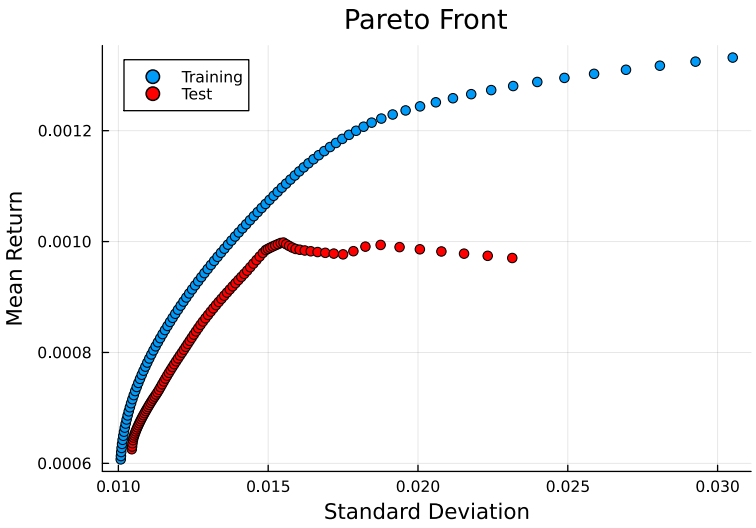

Figure 2: Train and test efficient frontier for an L2 regularised Markowitz model with a scenario-based standard deviation risk measure. The training set does much better than the test set, this is why other portfolio optimisation modalities have been invented.

2.2 Data flow

The library is configured to allow for different workflows. There is no rigid structure to the data flow. It is possible to use the library as simply as shown above or to build extremely complex workflows either manually, or using a pipeline. The functionality is grouped into three main stages, which break down into a hierarchy of standalone processes that can be used in isolation or as part of a larger workflow.

1. Preprocessing: data cleanup, filtering, and transformation.
   a. Transformation: fills out pathological data with something tractable.
   b. Preselection: uses various non-optimisation methods to preselect assets. Can operate at the price or returns stage.
   c. Returns computation: computes returns, and can impute, drop, or fill missing data.
2. Processing: these can be provided precomputed, which is valid for certain optimisers and standalone optimisations, or computed inside an optimisation, which is required for certain meta optimisers and for cross-validation. The only requirement to perform an optimisation is a prior, and for certain optimisers, a clustering.
   a. Prior statistics: standalone moment estimation, integrative returns distribution estimation, and adjustments to the moment estimations.
   b. Phylogeny: uncovers the relational structure of the asset universe through clustering and network analysis.

c. Constraint generation: lets users define constraints, some of which consume prior and phylogeny statistics.
d. Uncertainty sets: used for robust optimisation, they ameliorate the sensitivity to estimation errors.
e. Optimisation: consumes or internally carries out all the aforementioned steps and produces a result carrying every computed quantity.

3. Postprocessing: reporting and visualisation, which is by far the least mature aspect of the library.

The preprocessing and processing steps can be bundled into a single pipeline object, which is itself an estimator. That pipeline can be cross-validated and tuned as a unit, so the preprocessing and processing stacks are tuned together rather than one at a time.

2.3 Moments

PortfolioOptimisers.jl comes with a large number of moment estimation and prior statistics methods. It leverages StatsBase.jl's covariance estimator definitions and API to allow interoperability with the Julia ecosystem, for example by composing with CovarianceEstimation.jl. They can be combined in many ways: it is possible to compute the expected returns using a specific covariance method and vice-versa.

- Full or downside co-moments.
- Weighted, unweighted.
- Shrinkage [7].
- Probability-theoretic, including the Gerber statistic and its descendants [19], [20], [21].
- Denoising and detoning against the Marchenko-Pastur spectrum [11], [22].
- J-sparse, through information filtering networks [23].
- Regression-based.
- Rank based.
- Coskewness and cokurtosis.

2.4 Distance and phylogeny statistics

These bundle clustering and network analysis pipelines to explore the relational structure of the asset universe, and to derive insights that constraints and optimisers can consume. Clustering and network analysis are different, but they tackle similar issues from complementary angles, and the same analysis techniques exploit the structure that each uncovers. As such we choose to categorise them under the same umbrella. The distance matrices include the first and second

order distances defined in [11], and are compatible with every single covariance estimator that follows the StatsBase.jl API. The clustering and network analysis methods include:

- Distance, distance-of-distance and similarity matrices.
- Hierarchical agglomerative clustering.
- Direct bubble hierarchy trees [24], [25].
- K-means clustering.
- Minimum spanning trees.
- Planar maximally filtered graphs [26].
- Centrality and community detection.

2.5 Prior statistics

A prior is the returns distribution the optimisation actually sees, and it is the one component every optimiser requires. They fall under three categories:

- Frequentist: empirical and timeseries factor models, for low and high moments.
- Bayesian: Black-Litterman [27], in vanilla, factor, Bayesian and augmented forms.
- Information-theoretic: entropy pooling [28], including views on CVaR and EVaR [29], and linear or logarithmic opinion pooling [30].

2.6 Risk measures

A risk measure is an object, not a hard-coded branch. Any optimiser that takes one takes a vector of them, scalarised into a single expression by a sum, a maximum, a minimum, or a log-sum-exp. Every prior-derived quantity a measure needs, such as the expected returns, the covariance, the coskewness or the cokurtosis, may be given as a value or as the estimator that computes it. In the second case it is refitted against the optimisation's own prior, once per cross-validation fold and once per subset of a meta-optimiser, which a pasted matrix cannot do.

- Dispersion: variance, standard deviation, low order moments, range, Brownian distance variance, and a variance taken over an uncertainty set.
- Tail: value at risk, conditional value at risk [31], entropic value at risk [32], power norm and relativistic value at risk [10], and worst realisation. Most have a range twin, and the conditional ones have distributionally robust variants [33].
- Drawdown: the same tail family applied to the drawdown series [34], plus average drawdown, maximum drawdown, and the ulcer index.
- Ordered weights: Gini mean difference, tail Gini, and L-moments of arbitrary order, as disciplined convex programmes [35], [36], [37].

- High order: coskewness and cokurtosis, full or semi, standardised or not.
- Relative: tracking error, risk tracking error and turnover, which measure deviation from a reference portfolio rather than from zero.

Twelve further measures are exclusive to the hierarchical optimisers, which can use measures that are not convex in the weights because they never place them in a solver.

2.7 Uncertainty sets

Uncertainty sets are used for robust optimisation, where the parameters are only known to lie within a set. They ameliorate the sensitivity to estimation errors, and they currently cover the expected returns and the covariance.

- Box: the simplest set, defined by a lower and upper bound for each parameter [38].
- Ellipsoidal: defined by a radius and a covariance matrix, forming an ellipsoid in the parameter space [39].
- Characteristic: applies an L1 uncertainty set to the expected returns, which recovers the quintile portfolio as a special case [40]. This will eventually be generalised to any characteristic vector.

The bounds themselves come from a normal approximation, from a delta method, or from an ARCH bootstrap of the estimator.

2.8 Optimisers

PortfolioOptimisers.jl provides a large number of optimisers, in five families.

1. Naïve: equal weighted, inverse volatility, and random weighted. Speed and robustness, but not necessarily optimality. Interesting as sub-optimisers to more complex optimisers, as fallbacks, and as the benchmark that is hardest to beat out of sample [5].
2. JuMP.jl based [41]: the most flexible and powerful, but they require a solver, such as Clarabel [42], and are typically slower than first-order optimisers. They support a wide range of constraints and objectives, and they use conic reformulations throughout. The family covers mean risk, near optimal centering, risk budgeting [43] and its relaxed form, and factor risk contribution.
3. Hierarchical: these use the relational structure of the universe to compute the risk of each group and sub-group, producing a diversified portfolio that encodes the relational structure as well as the risk characteristics of each group. They are typically faster than JuMP.jl based optimisers and support fewer constraints, but they are very good for large universes.

The family covers hierarchical risk parity [44], hierarchical equal risk contribution [45], and Schur complementary allocation [46], which interpolates between hierarchical risk parity and minimum variance.

4. Meta-optimisers: these consume other optimisers and combine their results. They are typically slower, but can be very effective. The family covers nested clustering [11], stacking [47], and subset resampling [48].
5. Finite allocators: these do not optimise portfolios in the traditional sense. The user provides a finite cash amount and the asset prices, and the allocator computes the best portfolio attainable with that cash. They run after the others have produced a result.

2.9 Constraints, objectives, and penalties

There is a huge variety of constraints. Every optimiser supports weight bounds, but most of the richness is exclusive to JuMP.jl based estimators. Constraints are built by estimators rather than written as matrices, so a constraint is stated once and rebuilt against whatever data it meets. Linear constraints can be written as equations in near-natural language, for instance `"AAPL <= 0.1"`, `"MSFT >= AMD"`, or `"tech >= 0.15"` for a set of assets declared in the universe sets.

1. Weight bounds and budgets: the maximum and minimum weights, and the total value of the weights. Together they express leveraged, dollar-neutral, and long-short portfolios, and they integrate with the finite allocators, which adjust the available cash to the budget.
2. Linear constraints: used for relative weights, group exposures, risk contributions, and relational structure.
3. Cardinality constraints: sparsity, buy-in thresholds, and inclusion or exclusion rules, at the level of assets or of sets of assets. They are implemented as mixed integer linear constraints.
4. Fees and turnover: long and short proportional fees, fixed fees, and turnover fees, which penalise the returns and therefore, indirectly, the risk measures and the objective.
5. Weight penalties: L-norm penalties on the objective, or hard limits on the norm of the weights, which regulate sparsity and robustness.
6. Tracking: limits on how far a portfolio may drift from a reference, in weights or in risk.
7. Objectives: minimum risk, maximum return, maximum risk-adjusted return, and maximum utility, where the risk term may itself be a scalarised vector of risk measures.
8. Risk upper and return lower bounds: applicable simultaneously, and bounded by a scalar, a precomputed vector or range, or a frontier object that lets the optimiser compute the bounds on the fly. Bounding more than one quantity by a range produces a grid, and therefore a Pareto surface returned as a vector of results.

9. Custom constraints and objectives: user-defined terms that receive the optimiser's own context, and enter the objective as penalty contributions.
10. Time dependent: every constraint and every non-finite optimiser can be wrapped so that it varies across cross-validation folds. The wrapper takes a predefined list, or a function of the fold's context and the previous portfolio's weights, which allows a strategy to change its own constraints as it walks forward.

2.10 Cross-validation, hyperparameter tuning, and pipelines

The library comes equipped with a pipeline framework that can run an entire strategy end to end, and with a cross-validation framework that evaluates the out of sample performance of an optimisation or of a whole pipeline. The schemes include k-fold, index and date based walk-forward with purging, multiple randomised splits, and combinatorial splits that produce many backtest paths rather than one [49]. The same machinery drives the nested clustering and stacking meta-optimisers, which need out of sample predictions from their inner estimators before the outer optimisation can run.

The hyperparameter tuning framework is built on top of the cross-validation framework. Users address the steps of a pipeline by index or by name, and the fields to tune by name. Grid search takes grids of values, and random search takes grids, distributions from Distributions.jl [50], or both.

2.11 Design philosophy

PortfolioOptimisers.jl is implemented in Julia [51], using the JuMP.jl mathematical optimisation embedded language [41], and leverages other well-established Julia libraries for much of its functionality. Aside from a strong commitment to FOSS principles, the library is designed atop four pillars:

1. Well-defined type hierarchies. This lets us take advantage of generic fallbacks, and enables fearless extensibility, even at the user-level thanks to multiple dispatch.
2. Strongly typed, immutable structs. This lets the compiler aggressively optimise code, provides a single source of truth, and enables construction-time data validation that will not expire.
3. Compositional design. This lets us build complex objects from simpler ones, enables code reuse, and aids in extensibility.
4. Defensive programming. This lets us catch errors as early as possible, and provides a clear and consistent interface to the user.

With the exception of covariance estimators, which are defined in StatsBase.jl, every type in the library is a subtype of one of three types:

1. `AbstractEstimator`: these contain all the parameters needed to estimate a particular quantity, and are consumed by the functions that perform the estimation. They are the user-level vocabulary, they nest within each other, and they build more complex estimators.
2. `AbstractResult`: many estimators return more than one quantity, and those are returned in a result type. Estimators are configuration; results are data.
3. `AbstractAlgorithm`: these modify the behaviour of estimators via multiple dispatch. They are not used standalone, but as part of an estimator.

The strict adherence to a well-defined type hierarchy lets developers and users alike extend and modify the library's behaviour without modifying the source code. A new risk measure, estimator, or optimiser can live in a user's own package and still compose with everything described here.

3 Availability

PortfolioOptimisers.jl is registered in the Julia General registry, so `Pkg.add("PortfolioOptimisers")` is all that is needed, and it is available under an MIT license. It follows common software development best practices:

1. An extensive and high coverage test suite.
2. Automated documentation builds with a programmatically generated capability catalogue, an introductory user guide, deep examples, and fully documented public and private APIs.
3. Miscellaneous code quality checks.

All as part of a GitHub continuous integration pipeline. The listings in this paper are executed when it is built, so the paper is checked by the same standard as the rest of the project. The library is under active development, the roadmap is public, and contributions are welcome. We hope readers will find it a good place to run their own experiments.

Bibliography

[1] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[2] R. O. Michaud, "The Markowitz optimization enigma: Is 'optimized' optimal?," *Financial Analysts Journal*, vol. 45, no. 1, pp. 31–42, 1989.

[3] M. J. Best and R. R. Grauer, "On the sensitivity of mean-variance-efficient portfolios to changes in asset means: some analytical and computational results," *The Review of Financial Studies*, vol. 4, no. 2, pp. 315–342, 1991.

[4] V. K. Chopra and W. T. Ziemba, "The effect of errors in means, variances, and covariances on optimal portfolio choice," *The Journal of Portfolio Management*, vol. 19, no. 2, pp. 6–11, 1993.

[5] V. DeMiguel, L. Garlappi, and R. Uppal, "Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?," *The Review of Financial Studies*, vol. 22, no. 5, pp. 1915–1953, 2009.

[6] R. Jagannathan and T. Ma, "Risk reduction in large portfolios: Why imposing the wrong constraints helps," *The Journal of Finance*, vol. 58, no. 4, pp. 1651–1683, 2003.

[7] O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, 2004.

[8] V. DeMiguel, L. Garlappi, F. J. Nogales, and R. Uppal, "A generalized approach to portfolio optimization: Improving performance by constraining portfolio norms," *Management Science*, vol. 55, no. 5, pp. 798–812, 2009.

[9] P. N. Kolm, R. Tütüncü, and F. J. Fabozzi, "60 Years of portfolio optimization: Practical challenges and current trends," *European Journal of Operational Research*, vol. 234, no. 2, pp. 356–371, 2014.

[10] D. Cajas, *ADVANCED PORTFOLIO OPTIMIZATION*. Springer, 2025.

[11] M. M. L. De Prado, *Machine learning for asset managers*. Cambridge University Press, 2020.

[12] P. P. DANIEL, *Portfolio Optimization: Theory and Application*. CAMBRIDGE University Press, 2025.

[13] D. P. Palomar, "Portfolio Optimization." [Online]. Available: https://github.com/dppalomar

[14] D. Cajas, "Riskfolio-Lib (7.3)." [Online]. Available: https://github.com/dcajasn/Riskfolio-Lib

[15] H. Delatte, C. Nicolini, and M. Manzi, "skfolio." [Online]. Available: https://doi.org/10.5281/zenodo.16148630

[16] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[18] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[19] S. Gerber, H. Markowitz, P. Ernst, Y. Miao, P. Sargen, and others, "The Gerber statistic: A robust co-movement measure for portfolio optimization," *No and Sargen, Paul, The Gerber Statistic: A Robust Co-Movement Measure for Portfolio Optimization (July 4, 2021)*, 2021.

[20] W. Smyth and D. Broby, "An enhanced Gerber statistic for portfolio optimization," *Finance Research Letters*, vol. 49, p. 103229, 2022.

[21] S. Gerber, W. Smyth, H. Markowitz, Y. Miao, P. Ernst, and P. Sargen, "Squeezing financial noise: A novel approach to covariance matrix estimation," *Available at SSRN 4986939*, 2025.

[22] V. A. Marčenko and L. A. Pastur, "Distribution of eigenvalues for some sets of random matrices," *Mathematics of the USSR-Sbornik*, vol. 1, no. 4, p. 457, 1967.

[23] W. Barfuss, G. P. Massara, T. Di Matteo, and T. Aste, "Parsimonious modeling with information filtering networks," *Phys. Rev. E*, vol. 94, no. 6, p. 62306, Dec. 2016, doi: 10.1103/PhysRevE.94.062306.

[24] W.-M. Song, T. Di Matteo, and T. Aste, "Hierarchical information clustering by means of topologically embedded graphs," *PloS one*, vol. 7, no. 3, p. e31929, 2012.

[25] W.-M. Song, T. D. Matteo, and T. Aste, "Nested hierarchies in planar graphs," *Discrete Applied Mathematics*, vol. 159, no. 17, pp. 2135–2146, 2011, doi: https://doi.org/10.1016/j.dam.2011.07.018.

[26] G. P. Massara, T. Di Matteo, and T. Aste, "Network Filtering for Big Data: Triangulated Maximally Filtered Graph," *Journal of Complex Networks*, vol. 5, no. 2, pp. 161–178, 2016, doi: 10.1093/comnet/cnw015.

[27] F. Black and R. Litterman, "Global portfolio optimization," *Financial Analysts Journal*, vol. 48, no. 5, pp. 28–43, 1992.

[28] A. Meucci, "Fully flexible views: Theory and practice," *Risk*, vol. 21, no. 10, pp. 97–102, 2008.

[29] D. Cajas, "Entropy Pooling with CVaR and EVaR Views," *Available at SSRN 7120258*, 2026.

[30] C. Genest and J. V. Zidek, "Combining probability distributions: A critique and an annotated bibliography," *Statistical Science*, vol. 1, no. 1, pp. 114–135, 1986.

[31] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *Journal of Risk*, vol. 2, no. 3, pp. 21–41, 2000.

[32] A. Ahmadi-Javid, "Entropic value-at-risk: A new coherent risk measure," *Journal of Optimization Theory and Applications*, vol. 155, no. 3, pp. 1105–1123, 2012.

[33] P. Mohajerin Esfahani and D. Kuhn, "Data-driven distributionally robust optimization using the Wasserstein metric: performance guarantees and tractable reformulations," *Mathematical Programming*, vol. 171, no. 1, pp. 115–166, 2018.

[34] A. Chekhlov, S. Uryasev, and M. Zabarankin, "Drawdown measure in portfolio optimization," *International Journal of Theoretical and Applied Finance*, vol. 8, no. 1, pp. 13–58, 2005.

[35] D. Cajas, "OWA portfolio optimization: A disciplined convex programming framework," *Available at SSRN 3988927*, 2021.

[36] D. Cajas, "Higher order moment portfolio optimization with L-moments," *Available at SSRN 4393155*, 2023.

[37] D. Cajas, "Efficient Gini Mean Difference and Tail Gini Portfolio Optimization based on P-Norms," *Available at SSRN 4711326*, 2024.

[38] R. H. Tütüncü and M. Koenig, "Robust asset allocation," *Annals of Operations Research*, vol. 132, no. 1, pp. 157–187, 2004.

[39] D. Goldfarb and G. Iyengar, "Robust portfolio selection problems," *Mathematics of Operations Research*, vol. 28, no. 1, pp. 1–38, 2003.

[40] R. Zhou and D. P. Palomar, "Understanding the Quintile Portfolio," *IEEE Transactions on Signal Processing*, vol. 68, pp. 4030–4040, 2020, doi: 10.1109/TSP.2020.3006761.

[41] M. Lubin, O. Dowson, J. Dias Garcia, J. Huchette, B. Legat, and J. P. Vielma, "JuMP 1.0: Recent improvements to a modeling language for mathematical optimization," *Mathematical Programming Computation*, vol. 15, pp. 581–589, 2023, doi: 10.1007/s12532-023-00239-3.

[42] P. J. Goulart and Y. Chen, "Clarabel: An interior-point solver for conic programs with quadratic objectives." [Online]. Available: https://arxiv.org/abs/2405.12762

[43] S. Maillard, T. Roncalli, and J. Teiletche, "The properties of equally weighted risk contribution portfolios," *The Journal of Portfolio Management*, vol. 36, no. 4, pp. 60–70, 2010.

[44] M. L. De Prado, "Building diversified portfolios that outperform out of sample," *The Journal of Portfolio Management*, vol. 42, no. 4, pp. 59–69, 2016.

[45] T. Raffinot, "Hierarchical clustering-based asset allocation," *The Journal of Portfolio Management*, vol. 44, no. 2, pp. 89–99, 2017.

[46] P. Cotton, "Schur Complementary Allocation: A Unification of Hierarchical Risk Parity and Minimum Variance Portfolios." [Online]. Available: https://arxiv.org/abs/2411.05807

[47] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[48] W. Shen and J. Wang, "Portfolio selection via subset resampling," in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, 2017, pp. 1517–1523.

[49] M. M. L. De Prado, *Advances in financial machine learning*. John Wiley & Sons, 2018.

[50] M. Besançon *et al.*, "Distributions.jl: Definition and Modeling of Probability Distributions in the JuliaStats Ecosystem," *Journal of Statistical Software*, vol. 98, no. 16, pp. 1–30, 2021, doi: 10.18637/jss.v098.i16.

[51] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017, doi: 10.1137/141000671.